MaìLang：面向物联网与跨平台开发的 Rust 实现编程语言

MaìLang: A Rust-Implemented Programming Language for IoT and Cross-Platform Development

Maicarons
https://github.com/Maicarons/mailang

预印本（Preprint）
版本 0.4.0 · 2026

Abstract

物联网（IoT）与跨平台软件开发对编程语言提出了相互拉扯的需求：一方面需要足够小的运行时足迹与可预测的性能以适配 MCU 级设备，另一方面又需要现代语言特性（面向对象、模式匹配、错误处理、泛型）与多宿主嵌入能力。本文提出并实现 **MaìLang（麦语）**——一门以 Rust 编写、面向 IoT 与跨平台场景的现代编程语言。MaìLang 采用经典流水线架构（词法分析 → 语法分析 → 字节码编译 → 虚拟机）。编译器始终产出与后端无关的栈式字节码，而自 v0.4 起默认执行后端为寄存器（三地址）虚拟机：运行时一次性把栈字节码降为寄存器 IR，栈式后端则保留为可选后端。语言层面融合了类与继承、Trait、Rust 风格的 `Result/Option` 与 `?` 错误传播、模式匹配、字符串插值、UTF-8 原生标识符以及泛型函数 monomorphization；运行时层面实现了引用计数、环收集与标记清扫堆的混合垃圾回收，以及 `CallDirect`、尾调用优化（TCO）等热路径优化。跨平台能力通过 C FFI、WebAssembly、模拟 HAL 与多语言绑定演示覆盖 12 种宿主语言。实验表明：两种后端在全部 64 个已发布示例与库程序上逐字节一致，性能仅相差几个百分点（Fibonacci(30) 递归：栈式 119 ms、寄存器 116 ms，比值 0.97×），相对早期基线累计加速约 4.9×；在循环、函数调用与字符串插值等典型负载上优于 CPython 与 Node.js（V8）。集成测试套件在默认后端与 `--vm=stack` 两种配置下均为 137/137，全工作区 261 项测试全绿。本文系统描述 MaìLang 的语言设计、编译器与虚拟机实现、内存管理策略与跨平台嵌入方案，并诚实报告当前局限（无 `async`、泛型类字段擦除、IoT HAL 默认模拟等）。

关键词：编程语言实现；物联网；字节码虚拟机；Rust；跨平台嵌入；垃圾回收；monomorphization

1 引言

物联网（Internet of Things, IoT）设备与跨平台应用构成了当代系统软件的两大增长极。从运行在 ESP32-C3、Cortex-M4 等 MCU 上的传感器固件，到需要在浏览器、桌面与移动端共享逻辑的业务应用，开发者反复面临同一组张力：资源受限环境要求的精简运行时与现代应用开发要求的表达能力往往难以兼得。

现有方案各有取舍。C/C++ 与 Rust 提供极致性能与小足迹，但学习曲线陡峭、开发迭代慢；Python 与 JavaScript 生态丰富，但解释器体积与内存占用对 MCU 不友好；Lua 与 MicroPython 面向嵌入做了裁剪，却在类型表达、错误处理与工具链完整性上偏弱；WebAssembly 解决了分发问题，但其本身并不提供高级语言特性。换言之，**尚缺一门同时覆盖「现代语言特性 + 可控足迹 + 多宿主嵌入」的实用语言**。

MaìLang（麦语）正是针对这一缺口的设计与实现。文件扩展名为 `.mai`，实现语言为 Rust，核心目标包括：

1. **现代语言特性**：类与继承、Trait、闭包、模式匹配、`Result/Option` 与 `?`、字符串插值、UTF-8 标识符、泛型函数 monomorphization。
2. **可预测的解释执行**：栈式字节码 + slot locals（`CallDirect` + TCO）构成的字节码格式与后端无关，默认由寄存器（三地址）虚拟机执行，栈式虚拟机保留为可选后端。

3. **混合内存管理**：`Rc` 引用计数 + `collect_cycles()` 断环 + 标记清扫堆（`MarkSweepHeap`）。
4. **跨平台嵌入**：C ABI FFI、WebAssembly、模拟 HAL、多语言绑定，以及字节码产物 `.mailangbc`。
5. **工程完备性**：解析器错误恢复、分析器、LSP、格式化器、模块系统 v2、文件/静态 HTTP 包注册表。

本文贡献如下：

- 提出并实现 MaìLang 语言设计与完整实现流水线，全部以 Rust 工作区组织为 16 个 crate，覆盖词法、语法、分析、编译、虚拟机、GC、标准库、FFI、WASM、LSP 与包管理。
- 给出栈式/寄存器双后端的字节码虚拟机设计，并在两种后端上通过同一套 137 项集成测试与 64 个程序的逐字节比对验证特性对齐；默认后端自 v0.4 起为寄存器虚拟机。
- 实现面向 IoT 场景的混合 GC（引用计数 + 环断边 + 标记清扫）与一系列热路径优化（胖 `Value` 的 `Rc` 化、全局槽表、立即数融合、`CallDirect` 等），使 Fibonacci(30) 相对早期基线累计加速约 $4.9\times$（Phase F 阶段为 $4.25\times$）。
- 提供覆盖 C、Python、Node.js、Go、Java、Kotlin、C#、Swift、PHP、Ruby、Lua、TypeScript 共 12 种宿主的绑定演示与可复现基准，并诚实报告局限。

2 相关工作

嵌入式脚本语言。 Lua 是嵌入式脚本的经典范例，以极小运行时与 C API 友好著称 [1]，但其动态类型系统与弱错误处理模型难以支撑大型应用。MicroPython 将 Python 语义裁剪到 MCU，极大地降低了嵌入开发门槛 [2]，却同样受制于动态语义带来的隐式错误与有限的静态检查。QuickJS 是一款紧凑的 JavaScript 解释器，采用寄存器式字节码，在解释器性能上表现优异 [3]，但宿主被限定在 ECMAScript 语义内。

系统语言与安全运行时。 Rust 以所有权与生命周期在系统层兼顾安全与性能 [4]，是实现语言工具链的常见选择；MaìLang 将 Rust 用作实现语言，而非目标语言。Java 与 C# 证明了带 GC 的托管运行时在工业界的可行性 [5, 6]，但其运行时体积与启动成本对 MCU 场景并不友好。

WebAssembly 与跨语言嵌入。 WebAssembly 提供了可移植、沙箱化的分发格式 [7]，适合作为跨平台目标之一，但不替代语言前端与运行时语义。通过 C ABI 导出宿主函数注册接口是跨语言嵌入的稳妥路径 [8]，MaìLang 的 FFI 层即采用此设计，并由 `cbindgen` 自动生成头文件。

编译器与虚拟机技术。 栈式虚拟机实现简单、字节码紧凑，是教学与嵌入场景的常见选择；寄存器式虚拟机减少指令分派次数，在热点负载上更有优势 [9]。monomorphization 是 C++/Rust 实现零成本泛型的主流手段 [10]；MaìLang 将其用于泛型函数，并对泛型类采用字段擦除以控制实现复杂度。垃圾回收方面，引用计数无法单独处理环结构，通常需辅以环收集或追踪式回收 [11]——MaìLang 的混合方案正对应这一观察。

与上述工作相比，MaìLang 的定位是：**以现代静态风格语法组织 IoT/跨平台应用逻辑，以字节码解释器保证足迹可控，以 FFI/WASM/HAL 覆盖多宿主**，并提供完整的语言工具链与包管理闭环。

3 语言设计

3.1 概览与语法

MaìLang 源文件扩展名为 `.mai`。代码 1 展示了基本语法：函数、类型标注、字符串插值与 UTF-8 标识符。

定义 1 (MaìLang 程序)**.** 一个 *MaìLang* 程序是若干语句（`Stmt`）构成的序列，经词法分析得到 `Token` 流，由递归下降 + *Pratt* 解析器构造抽象语法树（`Program`/`Stmt`/`Expr`/`Pattern`），再编译为分块字节码（`Chunk`/`Bytecode`）供虚拟机执行。

Listing 1: MaìLang 基本语法示例

```
// hello.mai
fn greet(name: str) -> str {
    return "你好，{name}! "
}

println(greet("MaiLang"))
```

Listing 2: 类继承与 Trait

```
class Animal {
    let name: str
    fn init(name: str) { this.name = name }
    fn speak() -> str { return "{this.name} speaks" }
}

class Dog extends Animal {
    override fn speak() -> str {
        return "{this.name} barks!"
    }
}

trait Printable {
    fn to_string() -> str
    fn print() { println(this.to_string()) }
}
```

3.2 变量、函数与控制流

变量通过 `let`（不可变）、`var`（可变）与 `const`（常量）声明，编译器强制可变性——对 `let/const` 赋值将在语义分析阶段被拒绝。函数支持默认参数、递归、闭包捕获与尾调用优化。控制流包含 `if/elif/else`、`while`、`for` $\in$ range 与后缀 `expr match { }`。

3.3 面向对象与 Trait

MaìLang 支持类、单继承、构造函数（`init`）、`super()` 调用、`super.method()` 以及 `override`。Trait 通过 `implements` 挂到类上，支持默认方法与 `trait extends`（trait 继承）。代码 2 给出示意。

3.4 模式匹配与错误处理

模式匹配支持字面量、区间、`Ok/Err/Some/None`、或模式、守卫，以及数组/元组解构。错误处理采用 Rust 风格的 `Result<T,E>` / `Option<T>` 与 `?` 传播算子，见代码 3。

3.5 泛型

泛型函数采用 monomorphization：turbofish 语法 `id::<int>(x)` 将调用点特化为独立 chunk（命名 `id$int`），从而避免运行时类型擦除开销。泛型类目前采用字段擦除布局（单布局、字段动态），不进行按类型布局特化——这是有意的实现取舍，见第 6 节。

3.6 模块与包

模块系统 v2 采用「文件即模块」模型，`mailang.toml` 声明路径依赖（`mailang deps` 解析）。包注册表支持文件系统目录与静态 HTTP 镜像，提供 `publish/install/search/yank` 子命令。该设计刻意避免对中心化托管服务的依赖，以适配离线/内网 IoT 开发场景。

Listing 3: 模式匹配与 Result/Option

```
fn classify(n) {
    match n {
        0 => "zero"
        1..10 => "small"
        _ => "other"
    }
}

fn divide(a: float, b: float) -> Result<float, str> {
    if b == 0.0 { return Err("Division by zero") }
    return Ok(a / b)
}
```

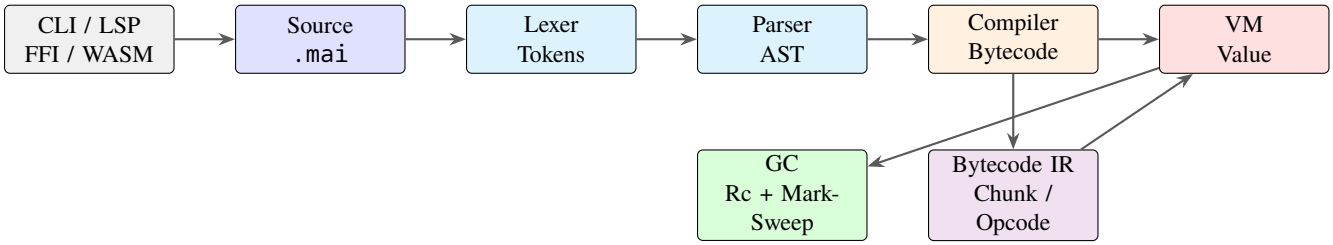


Figure 1: MaìLang 编译与执行流水线。上游为宿主集成层（CLI、LSP、FFI、WASM），下游为字节码 IR 与虚拟机，GC 跟踪运行时堆节点。

4 系统架构与实现

4.1 总体流水线

MaìLang 遵循经典编译器流水线，全部组件以 Rust crate 形式组织：

$$\text{Source (.mai)} \xrightarrow{\text{Lexer}} \text{Tokens} \xrightarrow{\text{Parser}} \text{AST} \xrightarrow{\text{Compiler}} \text{Bytecode} \xrightarrow{\text{VM}} \text{Value}$$

图 1 给出 crate 依赖与数据流。表 1 汇总各 crate 职责。

4.2 词法与语法分析

词法分析器对 UTF-8 标识符原生支持，并接受十六进制/八进制/二进制字面量与块注释。语法分析器采用递归下降处理语句与声明、Pratt 处理表达式优先级；关键特性是**解析错误恢复**（`recover_from_error` / `parse_program_recovering`），使 LSP 能在一次诊断中报告多条语法错误，而非在首个错误处终止。

4.3 字节码与编译器

字节码以 `Chunk` 为单位组织：每个函数对应一个 chunk，拥有独立的指令列表与常量池，主代码为 chunk 0。全局变量通过常量池名称解析，局部变量在编译期解析为栈槽位（slot），闭包经 upvalue 捕获。图 2 展示两种执行模型。

编译器在调用点区分两类函数调用：已知顶层函数且未被局部遮蔽时生成 `CallDirect`（跳过函数值占位槽），否则走通用 `Call`。对自递归尾调用复用栈帧实现 TCO。泛型函数在出现 turbofish 或唯一类型参数位置时触发 monomorphization，特化结果以 `name$type` 命名缓存。

4.4 虚拟机

编译器只产出一种中间表示——栈式字节码（局部变量在编译期解析为帧内 slot）。执行该表示的后端有两种：

Table 1: MaìLang Rust 工作区 crate 概览（共 16 个）。

Crate	阶段	职责
mailang-lexer	词法	Unicode 感知分词
mailang-ast	语法	AST 数据结构（含 serde）
mailang-parser	语法	递归下降 + Pratt；错误恢复
mailang-analyzer	语义	诊断 / 参数个数 / 简单类型
mailang-compiler	编译	字节码生成；泛型 monomorphization
mailang-bytecode	IR	栈式 IR + 寄存器 IR；`Value`
mailang-vm	执行	寄存器 VM（默认）+ 栈式 VM
mailang-gc	内存	断环 + `MarkSweepHeap`
mailang-stdlib	运行时	io/math/string/json/time/HAL
mailang-core	集成	Parser → Compiler → VM
mailang-cli	工具	run/eval/build/fmt/deps/registry/lsp
mailang-ffi	嵌入	C ABI；host 函数注册
mailang-wasm	嵌入	wasm-bindgen；`eval` / `eval_json`
mailang-lsp	工具	诊断/补全/跳转/悬停
mailang-module	包	模块 v2 + 路径依赖 + 注册表
mailang-macros	扩展	过程宏（当前为透传桩）

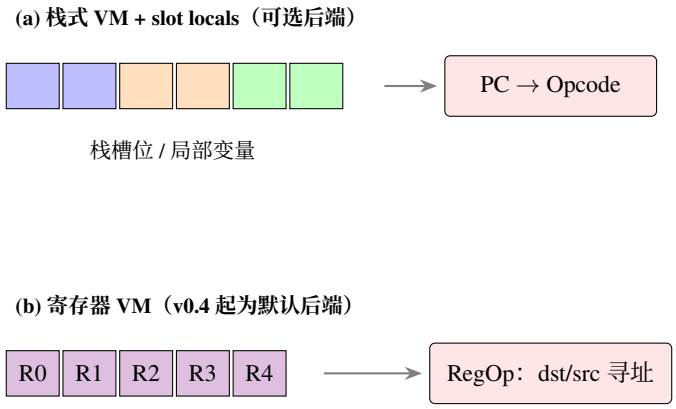


Figure 2: 两种虚拟机后端：(a) 栈式虚拟机（可选），局部变量解析为栈槽位；(b) 寄存器虚拟机（自 v0.4 起为默认），将栈操作降为 `RegOp` 的 dst/src 寄存器寻址。两后端共享同一份栈字节码，通过同一套 137 项集成测试与 64 个程序的逐字节比对。

- **寄存器（三地址）虚拟机：自 v0.4 起为默认后端**。运行前由 `StackToRegister` 一次性把栈 IR 降为寄存器 IR（`RegOp` 以 dst/src 寄存器寻址），每次 `run`/`eval` 只付一次降级成本，不改变持久化字节码；
- **栈式虚拟机（可选后端）**：操作数走显式操作数栈，局部变量按帧内 slot 索引访问。

两种后端始终一起编译，因此切换默认后端不改变二进制的构成。后端选择遵循确定性优先级：`--vm` 命令行参数优先于 `MAILANG_VM` 环境变量，后者优先于默认值（寄存器）；无法识别的环境变量取值一律回落到默认，以免游离变量破坏一次运行。`mailang repl` 同样接受 `--vm`。

运行时值 `Value` 是统一的动态类型枚举,覆盖 `Null`/`Bool`/`Int`/`Float`/`Str`/`Char`/`Array`/`Map`/`Tuple`/`Function`/`Clos` 等变体。堆对象经 `Rc`/`Rc<RefCell<…>>` 共享，避免深拷贝。

4.5 内存管理

单纯引用计数无法回收环结构（自引用实例、互相引用的数组等）。MaìLang 采用三层策略：

1. **日常分配/释放**：`Rc` 引用计数，热路径零额外开销；
2. **环断边**：`collect_cycles()` 从根出发 DFS，在灰集中发现回边时将其替换为 `Null`，使环可被引用计数释放；

3. **标记清扫**：`MarkSweepHeap` 维护堆节点普查表（heap census），从 VM 根（栈 / 全局 / upvalue）标记可达节点，清扫不可达节点并清除子边，从而真正回收环状垃圾。分配计数越过阈值时自动触发回收，阈值取 $\max(\text{MIN}, \text{live} \times 2)$——与存活集绑定的有界阈值。早期实现按 $\times 1.5$ 无上限递增阈值，在分配密集的循环中使堆占用线性增长（200 万/400 万次迭代分别达 151 MB/333 MB）；改为有界阈值后，同一负载的峰值工作集稳定在约 5 MB。

`gc_stats()` 暴露（tracked, collections, freed）统计，便于嵌入宿主观察回收行为。增量/并发 GC 尚未实现，见第 6 节。

4.6 性能优化

相对早期基线，MaìLang 实施了如下热路径优化：

- 堆对象 `Value` 改为 `Rc` 共享，消除深拷贝；
- 全局变量改为 `Vec` 槽表，`Load/StoreGlobal` 免哈希与字符串克隆；
- 数组/字典写操作原地修改；
- 自递归尾调用复用栈帧（TCO）；
- `CallDirect` 调用已知顶层函数，去掉函数占位槽；
- 算术/跳转立即数融合；
- 寄存器后端的取指循环把 `ip/chunk/base` 提升为循环局部变量、`Ret` 时截断寄存器文件、`BinImm/CmpImm` 整数快路径，使其 Fibonacci(30) 从 ∼1150 ms 降到与栈式后端持平；
- workspace release 采用 `lto` + `codegen-units=1` + `panic="abort"`。

4.7 跨平台与嵌入

C FFI。 `mailang-ffi` 导出稳定 C ABI，支持宿主函数注册与全局读写；头文件由 `cbindgen` 依据 `cbindgen.toml` 自动生成。特性开关 `std/gc` 映射为 C 宏 `MAILANG_STD/MAILANG_GC`。

WebAssembly。 `mailang-wasm` 经 wasm-bindgen 暴露 `eval` / `eval_json`，可在浏览器与 Node 中运行，并配有在线 Playground。

IoT HAL。 标准库提供 `gpio_*`、`adc_read`、`delay_ms` 等硬件抽象接口。**默认实现为宿主模拟**；真实硬件需通过 FFI 注册 HAL。交叉编译目标覆盖 `wasm32-unknown-unknown`、`thumbv7em-none-eabihf`（Cortex-M4）与 `riscv32imc-unknown-none-elf`（ESP32-C3）。

多语言绑定。 仓库提供 12 种宿主语言的绑定演示（C、C#、Go、Java、Kotlin、Lua、Node.js、PHP、Python、Ruby、Swift、TypeScript），验证 C ABI 的通用性。

4.8 工具链

CLI 提供 `run/eval/build/disasm/fmt/deps/registry/lsp` 以及交互式 `repl`；`run/eval/repl` 接受 `--vm` 选择执行后端，`disasm` 可打印编译器产出的栈字节码，或默认后端实际执行的寄存器降级形式。字节码可持久化为 `.mailangbc`，该产物与后端无关，同一份文件在两种后端下输出一致。LSP 基于 parser 错误恢复，支持多语法错误诊断与补全、跳转、悬停。分析器提供参数个数与简单类型诊断。

Table 2: 测试与双后端一致性。

执行配置	通过 / 总数	说明
默认后端（寄存器 VM）	137/137	`mailang run`，`MAILANG_VM` 未设
栈式 VM（可选后端）	137/137	`--vm=stack` / `MAILANG_VM=stack`
全工作区测试	261 全绿	含单元与集成
逐字节双后端比对	64/64 一致	`benchmark/compare_backends.ps1`

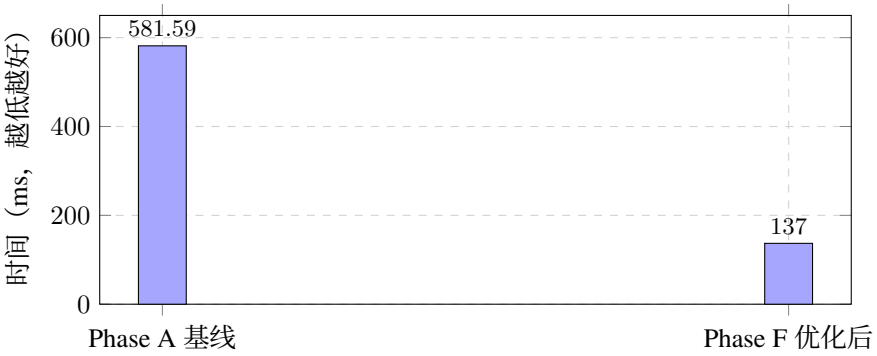


Figure 3: Fibonacci(30) 递归基准：Phase A 基线与 Phase F 优化后的耗时对比（release 构建，7 次平均）。加速比约 4.25×。

5 实验与评估

5.1 实验环境

所有性能数据来自 Windows 11 桌面环境（Intel CPU），Rust stable 工具链，`cargo build --release` 构建，release profile 启用 `lto` + `codegen-units=1` + `panic="abort"`。对比解释器包括 CPython 3.14、Node.js（V8 JIT）与 QuickJS（2025-04-26）。报告的 MaìLang 数据分两批：横向对比表与 Phase F 加速数据是 Phase L 时期以 7 次运行平均值采集的，v0.4 的双后端数据由 `benchmark/measure_fib.ps1` 以 3 次取最小值采集（口径不同，两批数字不宜直接相减）。项目当前版本为 0.4.0（Phase N 收尾 + 默认后端切换）。

5.2 正确性测试

集成测试套件共 137 项，覆盖算术、变量可变性、函数与默认参数、闭包、控制流、字符串插值、集合方法、模式匹配、OOP 与 Trait、`Result`/`Option` 与 `?`、模块导入、GC 回收路径，以及若干后端缺陷的回归用例（寄存器后端绕过宿主钩子、栈式后端直接调用帧的尾调用覆盖调用者槽位等）。如表 2 所示，**默认后端（寄存器 VM）与 `--vm=stack` 两种配置均 137/137 通过**，全工作区 261 项测试全绿。行为层面的对齐另有独立门禁：`benchmark/compare_backends.ps1` 把 `examples/` 与 `libs/` 下全部 66 个 `.mai` 程序在两种后端各跑一遍并逐字节比对输出（2 个因输出不确定或永不终止而跳过），当前 64/64 一致，任何新增差异都会使该门禁失败。

5.3 性能结果

递归热点：Fibonacci(30)。 在完成 F_1–F_5 热路径优化（Rc 胖 `Value`、热路径、立即数融合、`CallDirect` 无占位、`panic=abort`）后，Fibonacci(30) 递归（结果 832040）约需 **137 ms**（含约 11 ms 进程启动），相对 Phase A 基线 581.59 ms 加速约 4.25×，达到预定 3× 验收目标。图 3 展示加速前后对比。

v0.4 复测：双后端绝对性能。 v0.4 的默认后端是寄存器 VM。以 `benchmark/measure_fib.ps1` 采集（同一二进制，3 次取最小值）得：栈式 119 ms、寄存器 116 ms、不带 `--vm` 的默认运行 116 ms，寄存器对栈式比值为 0.97×——两者仅差几个百分点，远低于 2.0× 的验收门限；默认运行始终跟随寄存器线。同一台机器在负载较高时重跑一遍，三条线一起上浮（栈式 188 ms、寄存器 176 ms、默认 174 ms，比值

Table 3: 典型负载耗时对比（ms，7 次平均；越低越好）。QuickJS 列中「—」表示未测。

基准	MaìLang	CPython	Node.js	QuickJS
Fibonacci(30) 递归	581.6†	165.5	—	70.6
循环 sum 0..100k	35.4	105.2	53.8	9.8
函数调用 100k	45.6	100.9	51.2	16.0
字符串插值 10k	9.2	91.2	47.8	9.2
Fibonacci(30)（Phase F 优化后）	137.0	—	—	70.6
Fibonacci(30)（v0.4 默认后端）	**≈116–119**	—	—	70.6

† 为 Phase A 基线，与表中 Python/Node/QuickJS 同批测量（7 次平均）；后两行为 Phase F 与 v0.4 的单独复测，v0.4 为 3 次取最小值，机器负载较重时会升至 174–188 ms。

0.94×）：两后端的相对关系稳定，绝对数字随机器负载波动。以本批最快样本对 Phase A 基线 581.59 ms 计，累计加速约 5.0×（4.9–5.0×）；即便取负载较重那一次的采样，仍有 3.1–3.3×，即 3× 验收目标在两种后端与两次采样下均成立。

内存复测。 同一分配密集循环（`len([1,2,3])` 累加）在 200 万与 400 万次迭代下的峰值工作集为 5.1 MB（默认后端）与 4.9–5.1 MB（`--vm=stack`），不随迭代次数增长，门限为 64 MB。作为对照，有界阈值修复之前同一负载分别达到 151 MB 与 333 MB。

与 Python / Node.js / QuickJS 的对比。 表 3 给出四项典型负载的横向对比（越低越好）。可以看出：MaìLang 在循环、函数调用与字符串插值上均快于 CPython 与 Node.js；相对同为解释器的 QuickJS，字符串插值基本持平，循环与调用仍有差距，深度递归差距最大——这与解释器缺少内联缓存、无寄存器分配有关，也是后续优化方向。

相对优势的量化。 以 Phase A 基线口径计算：循环负载较 CPython 快约 3.0×、较 Node.js 快约 1.5×；函数调用较 CPython 快约 2.2×；字符串插值较 CPython 快约 9.9×、较 Node.js 快约 5.2×，与 QuickJS 持平。v0.4 的 Fibonacci(30)（116–119 ms）约为 QuickJS（70.6 ms）的 1.6–1.7×，而仍具备 CPython 之外的现代语言特性集合。

IoT 场景推算。 以 ESP32-C3 @ 160 MHz 相对现代 PC 慢 50–100× 的经验系数外推：循环 100k 约 1.8–3.5 s，字符串插值 10k 约 0.5–0.9 s。相较 MicroPython 的经验估计，MaìLang 在循环与字符串密集负载上预计有 2–10× 优势。需要强调：该推算**尚未在真实硬件上验证**，当前 IoT HAL 为模拟实现。

5.4 产物体积

字节码以 `.mailangbc` 形式持久化，便于嵌入宿主免源码分发；该产物与执行后端无关，同一份文件在两种后端下输出一致（CI 对同一 `.mailangbc` 的无参数运行与 `--vm=stack` 运行做 `diff` 断言）。体积最敏感的目标是 WebAssembly：`wasm32-unknown-unknown` 的 release 产物（`cargo build --release -p mailang-wasm`，未经 wasm-pack/wasm-opt 后处理）为 975,309 字节，而默认后端从栈式换成寄存器只增加 4 字节——两种后端始终一起编译，默认值本身不引入新代码，代价仅是每次运行多付一次栈到寄存器的降级。`mailang build examples/hello.mai` 产出的 `.mailangbc` 为 116 字节（1 个 chunk）。两个嵌入式目标 `thumbv7em-none-eabihf` 与 `riscv32imc-unknown-none-elf` 在 CI 中做 `no_std` 构建并报告 `mailang-bytecode` 的 rlib 体积（446.1 KiB / 447.3 KiB，含元数据，并非最终链接足迹）；目前只有构建检查，没有体积门限，也未在真实硬件上验证。

6 讨论与局限

无 `async`。 当前不提供 async/await 语法与运行时。对事件驱动的 IoT 固件，状态机式回调或协作式多任务是可接受的替代；对高并发服务端场景则构成明显短板。

泛型类字段擦除。 泛型函数已完成 monomorphization，但泛型类采用单一擦除布局，字段保持动态。这限制了字段访问的静态优化空间，也使完整泛型类型检查无法落实。

IoT HAL 默认模拟。 `gpio_*` 等接口默认由宿主模拟，真实硬件需 FFI 注册。本文报告的 IoT 数字为推算，不构成真实硬件验证声明。

GC 非增量。 Rc + 标记清扫可回收环，但回收过程非增量/并发，最坏延迟不可控。对硬实时 MCU 场景需进一步做分配预算或区域式内存策略。

双后端并存。 默认后端自 v0.4 起为寄存器 VM，栈式 VM 保留为可选后端。两者共享同一份字节码、同一套测试（137/137）与 64 个程序的逐字节比对，性能也基本持平（0.94–0.97×），因此双后端并存的收益是一条可回退的保守路径；代价是每个新特性都要在两条执行路径上实现并验证，这一长期维护成本是路线图中的显式条目。

注册表与宏。 包注册表仅支持本地/静态 HTTP，无公共托管服务；`mailang-macros` 仍为透传桩，过程宏体系尚未落地。

7 结论

本文提出了 MaìLang（麦语）——一门以 Rust 实现、面向 IoT 与跨平台开发的现代编程语言。通过「现代语言特性 + 字节码解释执行 + 混合 GC + 多宿主嵌入」的组合设计，MaìLang 在表达能力与运行时可控性之间取得了务实平衡。编译器只产出一种与后端无关的栈式字节码，寄存器（默认）与栈式（可选）两个后端在全部 137 项集成测试与 64 个已发布程序上完全一致，性能相差仅几个百分点；热路径优化使 Fibonacci(30) 相对早期基线累计加速约 4.9×；在循环、函数调用与字符串插值等典型负载上优于 CPython 与 Node.js。

未来工作包括：真实 IoT HAL 与硬件验证、增量 GC、泛型类 monomorphization、`async` 支持、双后端执行路径的持续一致性维护，以及公共包注册表服务。我们希望 MaìLang 能为「一门语言贯通 MCU 与云端」的工程实践提供可复用的参考实现。

致谢

本项目以 Apache-2.0 许可证开源。感谢 Rust 社区提供的工程基础设施，以及所有在语言设计讨论中给出反馈的同行。

References

[1] R. Ierusalimschy, L. H. de Figueiredo, and W. Celes, “Lua — an extensible extension language,” *Software: Practice and Experience*, vol. 26, no. 6, pp. 625–652, 1996.

[2] D. P. George and G. M. T. Group, “MicroPython: Python for microcontrollers,” *GitHub / micropython project*, 2014. https://micropython.org

[3] F. Bellard, “QuickJS: A small and embeddable JavaScript engine,” 2019. https://bellard.org/quickjs/

[4] N. D. Matsakis and F. S. Klock, “The Rust Programming Language,” *ACM SIGAda Ada Letters*, vol. 34, no. 3, pp. 103–104, 2014.

[5] J. Gosling, B. Joy, G. Steele, G. Bracha, and A. Buckley, *The Java Language Specification*, Java SE 21 ed. Oracle, 2023.

[6] E. Meijer and M. Torgersen, “Programming with circles, triangles and rectangles,” in *Proc. XML 2004 Conf.*, 2004.

[7] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Danilov, J. Bastien, and L. Chen, “Bringing the Web up to Speed with WebAssembly,” in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

[8] S. Chiba, “A metaobject protocol for C++,” in *Proc. ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 1995, pp. 285–299.

[9] W. M. McKeeman, “Peephole optimization,” *Communications of the ACM*, vol. 8, no. 7, pp. 443–444, 1965.

[10] B. Stroustrup, *The Design and Evolution of C++*. Addison-Wesley, 1994.

[11] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Press, 2016.

[12] R. Nystrom, *Crafting Interpreters*. Gene Benioff, 2021.

[13] A. W. Appel and M. Ginsburg, *Modern Compiler Implementation in C*. Cambridge University Press, 1998.

[14] Rust Project Developers, “The Rustonomicon: FFI,” 2024. https://doc.rust-lang.org/nomicon/ffi.html

[15] Maicarons, “MaìLang: A modern programming language for IoT and cross-platform development,” 2026. https://github.com/Maicarons/mailang